Task 1 – Digital Adder

Introduction

This laboratory exercise focused on designing, simulating, and implementing an 8-bit ripple-carry adder using VHDL and the Vivado design environment. The work began with a 1-bit full adder, which was extended to an 8-bit RCA where the carry signal propagates from the LSB to the MSB. The design was verified through behavioural simulation using structured testbench stimuli, and then mapped onto the MODSYS 2.0 FPGA board for hardware testing. Finally, timing analysis of the synthesized and implemented designs provided insight into the propagation delay characteristics of ripple-carry architectures.

Task 1.1 – Full Adder Cell

A full adder is a fundamental combinational circuit used to add two binary digits A and B together with a carry input c_i. It produces two outputs: the sum bit S and the carry-out bit c_o. The full adder is the basic building block for multi-bit adders such as the ripple-carry adder implemented later in this exercise.

Truth Table

Table 1 shows the complete truth table of the 1-bit full adder.

c_i	B	A	c_o	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Table 1: Truth table of the 1-bit full adder

Boolean Equation Derivation

From the truth table, the canonical Sum-of-Products (SOP) expressions for S and c_o can be derived. For the sum bit:

$$S = A \oplus B \oplus c_i$$

This expression results directly from the fact that the sum bit is 1 whenever an *odd* number of input bits are 1.

The carry-out becomes 1 whenever at least two of the inputs are 1. The minimized Boolean equation is:

$$c_o = (A \cdot B) + (A \cdot c_i) + (B \cdot c_i)$$

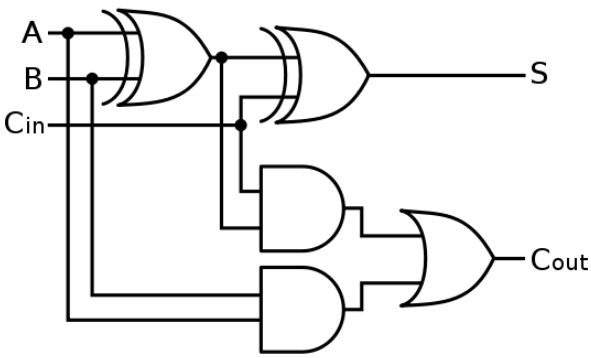

Figure 1: Gate-level schematic of the 1-bit full adder

VHDL Implementation (Full Adder)

VHDL Design and Test Bench Code The complete VHDL implementation for the full adder is provided in Appendix A.

Task 1.2 – Ripple-Carry Adder

The ripple-carry adder (RCA) extends the 1-bit full adder from Task 1.1 to an 8-bit arithmetic unit. Eight full adder cells are connected in sequence, where each carry output feeds the carry input of the next higher bit position. As a result, the carry "ripples" through the chain from the least significant bit (LSB) to the most significant bit (MSB).

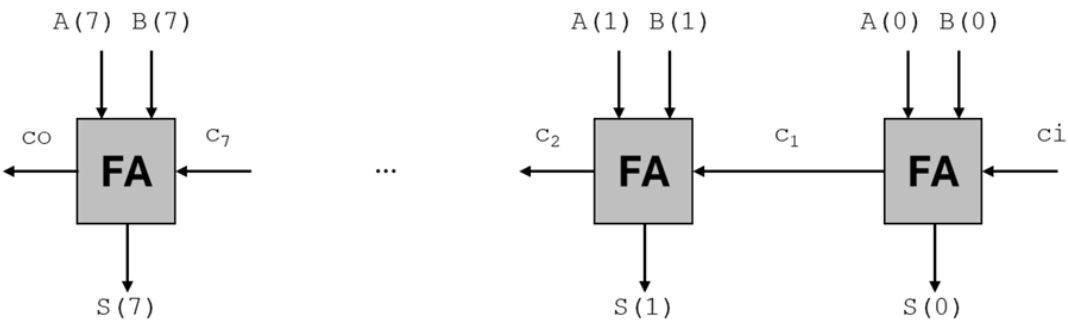

Figure 2: Enter Caption

Testbench Behaviour

In the provided `adder_tb.vhd` the stimuli are applied via a constant array `TEST_DATA`. Every entry contains:

- operand A (8 bit),
- operand B (8 bit),
- carry input c_i,
- expected sum result S,
- expected carry output c_o.

In the stimuli process a clock-like signal `cycle` is toggled. On every rising edge. At least five additional meaningful test vectors were added. They cover:

- simple additions without carry,
- additions with carry propagation over several bits,
- overflow situations (e.g. `11111111 + 00000001`),
- mixed patterns of high and low bits,
- edge cases around the MSB transition.

RCA Implementation

The RCA was implemented in `adder.vhd` as a combinational process using a `for`-loop. For each bit position i the sum and the next carry are calculated according to the full-adder equations

$$S_i = A_i \oplus B_i \oplus c_i, \qquad c_{i+1} = A_i B_i + A_i c_i + B_i c_i.$$

A schematic view of the synthesized adder in Vivado is shown in Figure 3.

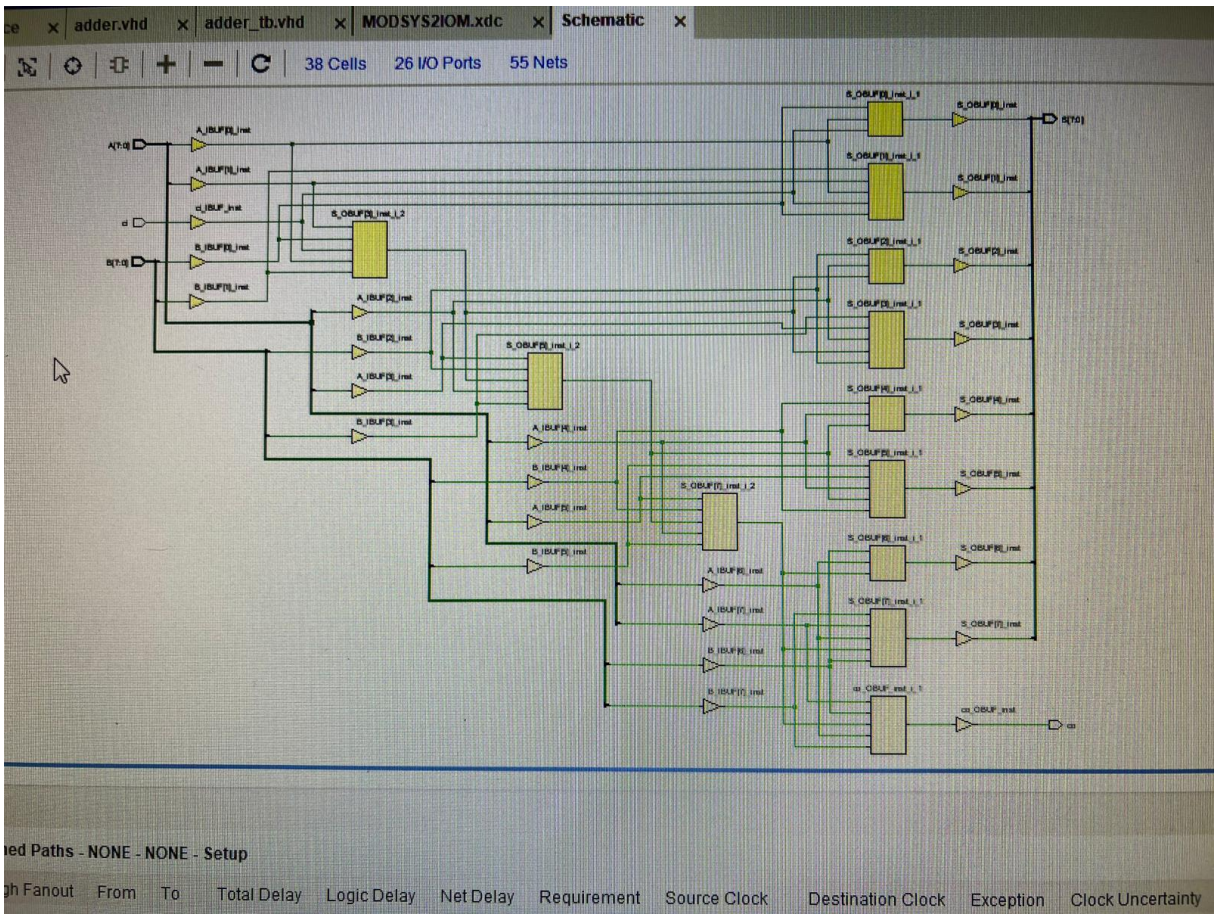

Figure 3: Vivado schematic of the 8-bit ripple–carry adder

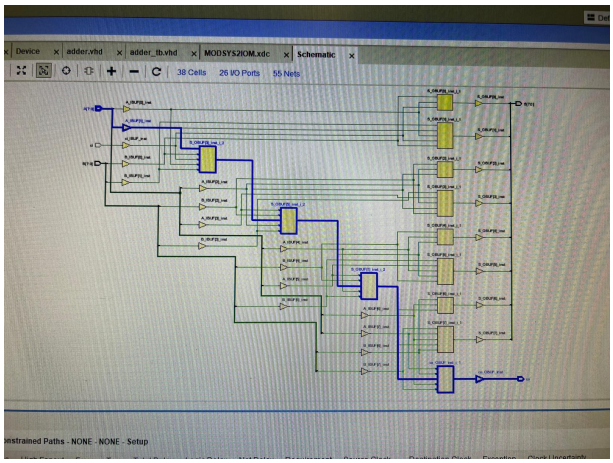

Figure 4: Detailed schematic view of the RCA signal connections

Behavioural Simulation

The behavioural simulation was run in Vivado. All test vectors in `TEST_DATA` produced the expected outputs, i.e. no assertion warnings occurred. The waveform confirms that the carry propagates correctly through all 8 stages.

Task 1.3 – Constraints and Timing Analysis for the Adder

The constraint file `MODSYS2IOM.xdc` was adapted to map all logical VHDL ports of the adder to the physical pins of the MODSYS 2.0 IOM boards. The switches on Connector 3 and Connector 4 were assigned to the input operands $A[7:0]$ and $B[7:0]$, while the rightmost push button on Connector 3 was mapped to the carry-in signal `ci`. The LEDs on Connector 3 were configured to display the sum bits $S[7:0]$ and the final carry-out `co`, enabling direct hardware testing of the adder on the FPGA board.

Timing Analysis

The timing summary was analysed for both the synthesized design and the implemented (placed and routed) design.

Logic Synthesis (pre-route) In the synthesis timing report the paths consist only of logic elements (LUTs and internal nets). The critical paths listed correspond to the carry chain of the RCA, which is expected because every bit depends on the previous carry.

Physical Implementation (post-route) After implementation the timing report also includes routing delays. The critical path delay increases because the signals now travel across the FPGA routing resources. Nevertheless, the longest path remains the carry propagation from the LSB to the MSB.

In summary, the timing analysis demonstrates the typical behaviour of a ripple–carry adder: the maximum delay is dominated by the serial carry propagation through all bit positions.

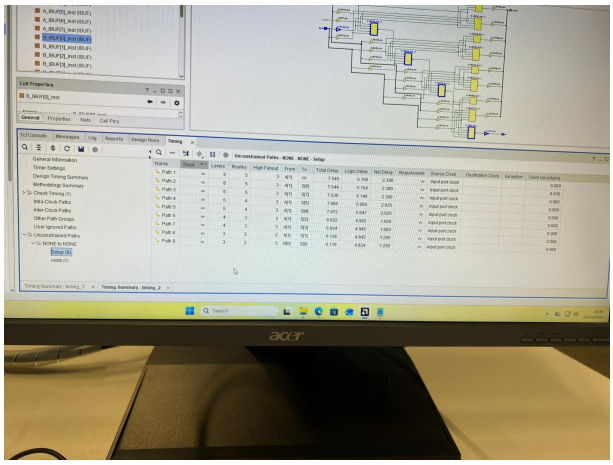

Figure 5: Timing summary of the implemented RCA design

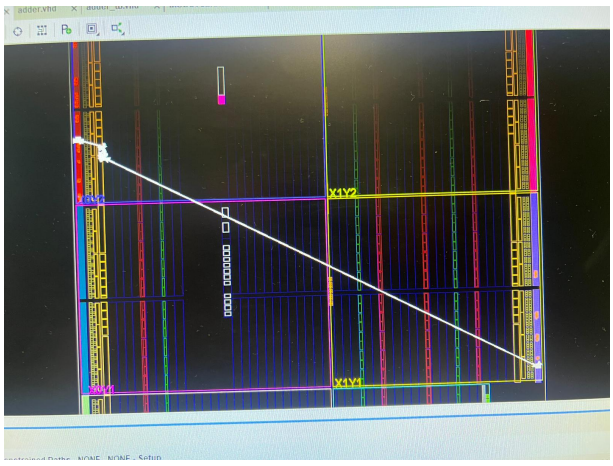

Figure 6: Routed critical path of the RCA on the FPGA device

VHDL Implementation

The complete VHDL implementation for the full adder is provided in Appendix B.

Conclusion

The experiment demonstrated the full design flow of an 8-bit ripple-carry adder, from VHDL modelling to FPGA implementation. Both simulation and hardware testing confirmed correct functionality. Timing analysis showed the expected limitation of ripple-carry adders, where delay increases with the carry chain.

Task 2 – Addsub Module

Introduction

This task extends the 8-bit adder to an Addsub unit capable of both addition and subtraction using a select input. Two's-complement logic and status flags (C, V, N, Z) are added, simulated, and implemented on the MODSYS 2.0 FPGA board.

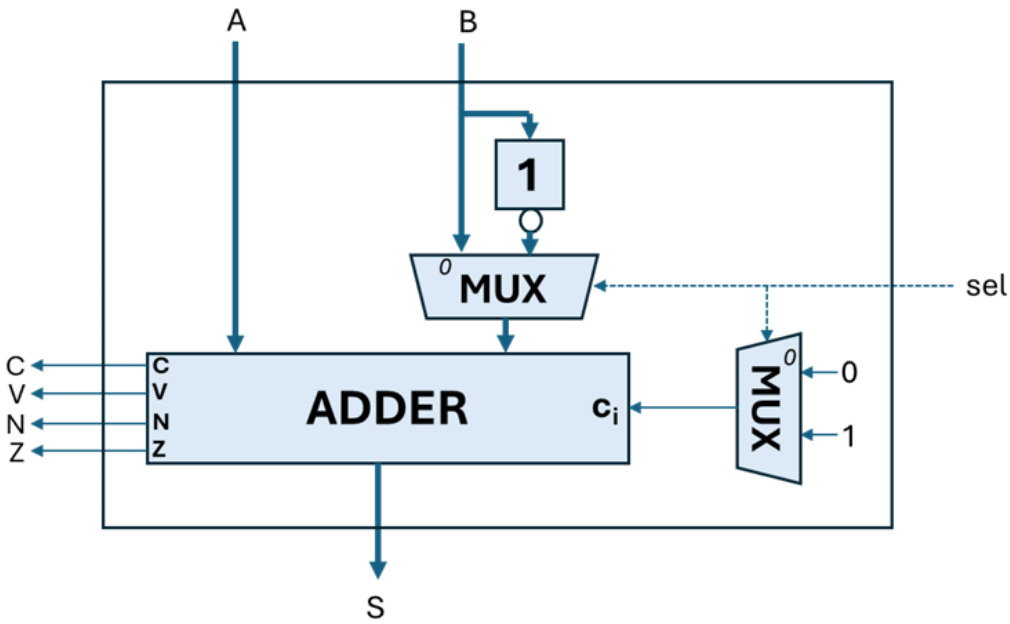

Figure 7: Block diagram of the Addsub module showing the MUX-based selection between addition and subtraction and the generation of the status flags.

Preparation

Before implementing the Addsub module, several design steps were required:

- **Multiplexer Integration**: The MUX selects between operand B and its inverted form. Since the process already includes `A, B, sel` in its sensitivity list, **no change to the sensitivity list is required**.
- **Testbench Changes**: Compared to `adder_tb`, the Addsub testbench includes:
  - the additional control input `sel`,
  - extended stimulus records containing C, V, N, Z,
  - assertions for all status flags,
  - new test vectors for addition, subtraction, and flag verification.
- **Additional Test Vectors**: At least five new vectors were added to cover:
  - unsigned overflow,
  - subtraction underflow,

  - signed overflow,
  - zero-result detection,
  - mixed signed/unsigned edge cases.

- **Constraint Extension**: The ports `sel`, `C`, `V`, `N`, `Z` were mapped to buttons and LEDs on the MODSYS 2.0 IOM boards for hardware validation.

Four status flags were added:

- **C**: unsigned carry flag,
- **V**: signed overflow flag $V = co \oplus c_7$,
- **N**: negative result $N = S(7)$,
- **Z**: zero-detection flag.

Task 2.1 – Addsub Design

The Addsub module extends the 8-bit ripple-carry adder from Task 1 by enabling both addition and subtraction using a select input `sel`. When `sel = 0`, the circuit performs normal addition. When `sel = 1`, operand B is transformed into its two's complement, allowing subtraction to be implemented using the same adder hardware.

This transformation is achieved by:

- bitwise negation of operand B,
- forcing the carry-in signal of the adder to logic 1.

A multiplexer selects whether B or its inverted version is fed into the adder. A second MUX switches the carry-in between `0` and `1`.

Schematic Interpretation

Figure 7 illustrates the complete Addsub architecture:

- Operand A is fed directly into the adder.
- Operand B passes through a MUX that selects between:
  - its normal form (`sel=0`),
  - its inverted form for subtraction (`sel=1`).
- A second MUX controls the initial carry:
  - `0` for addition,
  - `1` for subtraction.
- The adder computes the result and outputs the status flags.

This diagram helps visualizing how subtraction is achieved without building a second arithmetic unit.

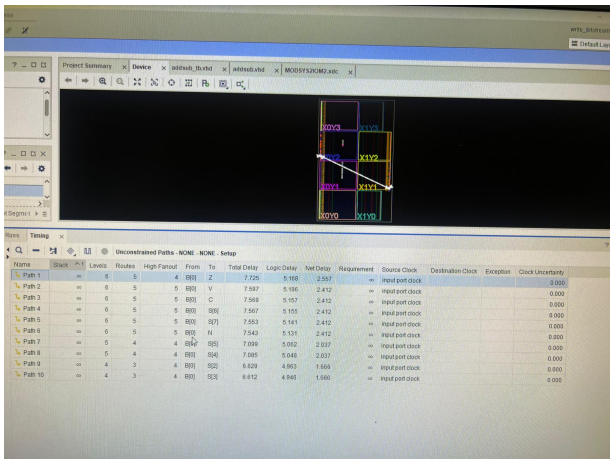

Figure 8: Behavioural simulation

Task 2.2 – Testbench Behaviour

The Addsub testbench (`addsub_tb.vhd`) extends the adder testbench from Task 1 with additional functionality:

- a new control input `sel`,
- extended stimulus record including:
  - expected sum,
  - expected C, V, N, Z flags.
- assertions for each of the status flags,
- additional test vectors covering both addition and subtraction.

Five new meaningful test cases were added to validate:

- subtraction producing a negative value,
- unsigned overflow in addition,
- signed overflow in subtraction,
- zero-result detection,
- MSB sign transitions.

A behavioural simulation confirmed that the status flags update correctly for each arithmetic operation.

Constraints and Timing Analysis

The constraint file from Task 1 was extended to support the new ports:

- **sel** mapped to a button,
- **C, V, N, Z** mapped to the LEDs of the MODSYS 2.0 board,

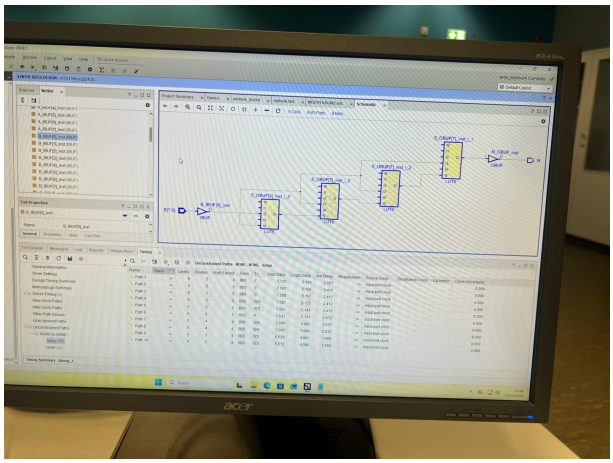

Figure 9: FPGA device layout

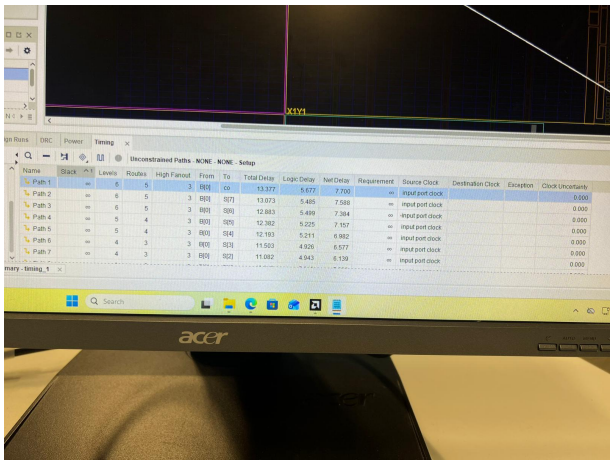

Figure 10: ost-implementation timing summary of the Addsub module.

- sum output $S[7:0]$ mapped as before.

This allows interactive switching between addition and subtraction and real-time observation of the arithmetic status flags.

Timing Analysis

Timing analysis after implementation revealed that:

- the critical path increased slightly compared to Task 1,
- the added MUX and flag logic introduce additional LUT levels,
- the ripple-carry chain still dominates the total delay.

VHDL Implementation The complete Addsub VHDL design is provided in Appendix C.

Appendix

A Full Adder (full_adder.vhd)

Listing 1: Full Adder Design

```
library ieee;
use ieee.std_logic_1164.all;

entity full_adder is
    port (
        A   : in  std_logic;
        B   : in  std_logic;
        ci  : in  std_logic;
        S   : out std_logic;
        co  : out std_logic
    );
end entity full_adder;

architecture behavioral of full_adder is
begin
    S  <= A xor B xor ci;
    co <= (A and B) or (A and ci) or (B and ci);
end architecture behavioral;
```

Listing 2: Full Adder Testbench

```
library ieee;
use ieee.std_logic_1164.all;

entity fa_tb is
end entity fa_tb;

architecture tb of fa_tb is

    signal A  : std_logic := '0';
    signal B  : std_logic := '0';
    signal ci : std_logic := '0';

    signal S  : std_logic;
    signal co : std_logic;

begin
    UUT: entity work.full_adder
        port map (
            A  => A,
            B  => B,
            ci => ci,
            S  => S,
```

```
            co => co
        );

    stim_proc: process
    begin
        A <= '0'; B <= '0'; ci <= '0'; wait for 10 ns;
        A <= '0'; B <= '0'; ci <= '1'; wait for 10 ns;
        A <= '0'; B <= '1'; ci <= '0'; wait for 10 ns;
        A <= '0'; B <= '1'; ci <= '1'; wait for 10 ns;
        A <= '1'; B <= '0'; ci <= '0'; wait for 10 ns;
        A <= '1'; B <= '0'; ci <= '1'; wait for 10 ns;
        A <= '1'; B <= '1'; ci <= '0'; wait for 10 ns;
        A <= '1'; B <= '1'; ci <= '1'; wait for 10 ns;

        wait;
    end process;

end architecture tb;
```

B 8-bit Ripple-Carry Adder (adder.vhd)

Listing 3: Adder

```
library IEEE;
use IEEE.std_logic_1164.all;

entity adder is
    port(
        A     : IN  std_logic_vector(7 downto 0);
        B     : IN  std_logic_vector(7 downto 0);
        ci    : IN  std_logic;
        S     : OUT std_logic_vector(7 downto 0);
        co    : OUT std_logic
        );
end entity;

architecture rtl of adder is
begin

adding: process(A, B, ci)

variable c_v : std_logic;
variable S_v : std_logic_vector(7 downto 0);
variable A_v, B_v : std_logic_vector(7 downto 0);

begin
    A_v := A;
    B_v := B;
    c_v := ci;
```

```
    for i in 0 to 7 loop
        S_v(i) := A_v(i) xor B_v(i) xor c_v;
        c_v := (A_v(i) and B_v(i)) or
               (A_v(i) and c_v) or
               (B_v(i) and c_v);
    end loop;

    S <= S_v;
    co <= c_v;

end process;

end architecture rtl;
```

Listing 4: Full Adder Testbench

```
library IEEE;
use IEEE.std_logic_1164.all;

entity adder_tb is
end entity;

architecture behavioral of adder_tb is

signal cycle  : std_logic :='0';
signal A_s, B_s : std_logic_vector(7 downto 0);
signal S_s    : std_logic_vector(7 downto 0);
signal ci_s   : std_logic;
signal co_s   : std_logic;

constant PERIOD : time := 100 ns;
constant OFFSET : time := 20 ns;

component adder is
    port( A : in std_logic_vector(7 downto 0);
          B : in std_logic_vector(7 downto 0);
          ci: in std_logic;
          S : out std_logic_vector(7 downto 0);
          co: out std_logic );
end component;

type stimulus_response_type is record
    operand_A  : std_logic_vector(7 downto 0);
    operand_B  : std_logic_vector(7 downto 0);
    carry_in   : std_logic;
    result_S   : std_logic_vector(7 downto 0);
    carry_out  : std_logic;
end record;

type stimulus_response_array_type is array (natural range
   <>) of stimulus_response_type;
```

```
constant TEST_DATA: stimulus_response_array_type := (
    ("00000000","00000001",'0', "00000001",'0'),
    ("00000001","00000001",'0', "00000010",'0'),
    ("00000000","00000001",'1', "00000010",'0'),
    ("00000010","00000010",'0', "00000100",'0'),
    ("11111111","00000001",'0', "00000000",'1')
);

begin

cycle <= not cycle after PERIOD/2;

dut: adder
    port map(A => A_s, B => B_s, S => S_s, ci => ci_s, co =>
       co_s);

stimuli: process
begin
    wait for PERIOD;
    wait for OFFSET;

    for I in TEST_DATA'range loop
        wait until rising_edge(cycle);

        A_s <= TEST_DATA(I).operand_A;
        B_s <= TEST_DATA(I).operand_B;
        ci_s <= TEST_DATA(I).carry_in;

        wait for OFFSET;

        assert S_s = TEST_DATA(I).result_S report "error in
           sum" severity warning;
        assert co_s = TEST_DATA(I).carry_out report "error
           in carry" severity warning;

    end loop;

    wait;
end process;

end architecture behavioral;
```

Listing 5: constraints

```
#####################################################
# Digital Circuits     Exercise 2
## Ripple-Carry Adder (Important Constraints Only)
#####################################################
```

```
##############
# CONNECTOR 3
# Button     ci
set_property PACKAGE_PIN G16 [get_ports {ci}];
set_property IOSTANDARD LVCMOS33 [get_ports {ci}];

##############
# CONNECTOR 3
# Switches     A[7:0]
set_property PACKAGE_PIN G18 [get_ports {A[0]}];
set_property PACKAGE_PIN B11 [get_ports {A[1]}];
set_property PACKAGE_PIN A11 [get_ports {A[2]}];
set_property PACKAGE_PIN D15 [get_ports {A[3]}];
set_property PACKAGE_PIN C15 [get_ports {A[4]}];
set_property PACKAGE_PIN A13 [get_ports {A[5]}];
set_property PACKAGE_PIN A14 [get_ports {A[6]}];
set_property PACKAGE_PIN F15 [get_ports {A[7]}];

set_property IOSTANDARD LVCMOS33 [get_ports {A[*]}];

##############
# CONNECTOR 3
# LEDs     S[7:0] + co
set_property PACKAGE_PIN H15 [get_ports {S[0]}];
set_property PACKAGE_PIN K13 [get_ports {S[1]}];
set_property PACKAGE_PIN J13 [get_ports {S[2]}];
set_property PACKAGE_PIN K15 [get_ports {S[3]}];
set_property PACKAGE_PIN J15 [get_ports {S[4]}];
set_property PACKAGE_PIN E16 [get_ports {S[5]}];
set_property PACKAGE_PIN E15 [get_ports {S[6]}];
set_property PACKAGE_PIN H17 [get_ports {S[7]}];

set_property PACKAGE_PIN D17 [get_ports {co}];

set_property IOSTANDARD LVCMOS33 [get_ports {S[*]}];
set_property IOSTANDARD LVCMOS33 [get_ports {co}];

#####################################################
# CONNECTOR 4
# Switches     B[7:0]
set_property PACKAGE_PIN R7 [get_ports {B[0]}];
set_property PACKAGE_PIN R5 [get_ports {B[1]}];
set_property PACKAGE_PIN R6 [get_ports {B[2]}];
set_property PACKAGE_PIN N4 [get_ports {B[3]}];
set_property PACKAGE_PIN M4 [get_ports {B[4]}];
set_property PACKAGE_PIN T3 [get_ports {B[5]}];
set_property PACKAGE_PIN R3 [get_ports {B[6]}];
set_property PACKAGE_PIN U7 [get_ports {B[7]}];

set_property IOSTANDARD LVCMOS33 [get_ports {B[*]}];
```

```
#####################################################
```

C Addsub Unit (addsub.vhd)

Listing 6: Addsub – Adder/Subtractor Design File

```
-----------------------------------------------------------------
--
-- [IE3-DI] Digital Circuits Winter Term 2025
--          Exercise 2
--
-- @name:   addsub.vhd
-- @author: Sharon and Robin
-- @description: 8-bit Adder/Subtractor Unit with Status
   Flags
--               DESIGN FILE
--
-- (c) 2025 HAW Hamburg
--
-----------------------------------------------------------------

---------------------------------
-- libraries / packages
library IEEE;
use IEEE.std_logic_1164.all;

---------------------------------
-- entity
entity addsub is
    port(
        A    : IN  std_logic_vector(7 downto 0);   --
           Operand A (8 bit)
        B    : IN  std_logic_vector(7 downto 0);   --
           Operand B (8 bit)
        sel  : IN  std_logic;                      --
           Select: 0 = add, 1 = subtract

        S    : OUT std_logic_vector(7 downto 0);   -- Sum
        C    : OUT std_logic;                      -- Carry
           flag (unsigned overflow)
        V    : OUT std_logic;                      --
           Overflow flag (signed)
        N    : OUT std_logic;                      --
           Negative flag
        Z    : OUT std_logic                       -- Zero
           flag
        );
end entity;
```

```
---------------------------------
-- architecture
architecture rtl of addsub is
begin

---------------------------------
-- combinatorial process
adding: process(A, B, sel)

-- variable declarations
variable c_v : std_logic;
variable carry_into_msb : std_logic;
variable S_v : std_logic_vector(7 downto 0);
variable A_v, B_operand : std_logic_vector(7 downto 0);

begin
    A_v := A;

    -- Select operation: add or subtract
    if sel = '1' then
        B_operand := not B;
        c_v := '1';
    else
        B_operand := B;
        c_v := '0';
    end if;

    -- Ripple-carry adder / subtractor
    for i in 0 to 7 loop
        S_v(i) := A_v(i) xor B_operand(i) xor c_v;

        if i = 6 then
            carry_into_msb :=
                (A_v(i) and B_operand(i)) or
                (A_v(i) and c_v) or
                (B_operand(i) and c_v);
        end if;

        c_v :=
            (A_v(i) and B_operand(i)) or
            (A_v(i) and c_v) or
            (B_operand(i) and c_v);
    end loop;

    S <= S_v;
    C <= c_v;

    -- Status flags:
    V <= c_v xor carry_into_msb;  -- Signed overflow
    N <= S_v(7);                  -- Negative
    Z <= '1' when S_v = "00000000" else '0';
```

```
end process;

end architecture rtl;
```

Listing 7: Addsub – Testbench

```
-----------------------------------------------------------------
--
-- [IE3-DI] Digital Circuits Winter Term 2025
--          Exercise 2
--
-- @name:   addsub_tb.vhd
-- @description: Adder/Subtractor Unit with Status Flags
--               TESTBENCH
--
-----------------------------------------------------------------

library IEEE;
use IEEE.std_logic_1164.all;

entity addsub_tb is
end entity;

architecture behavioral of addsub_tb is

signal cycle        : std_logic := '0';
signal A_s, B_s     : std_logic_vector(7 downto 0);
signal S_s          : std_logic_vector(7 downto 0);
signal sel_s        : std_logic;
signal C_s, V_s, N_s, Z_s : std_logic;

constant PERIOD : time := 100 ns;
constant OFFSET : time := 20 ns;

component addsub
    port(
        A   : IN  std_logic_vector(7 downto 0);
        B   : IN  std_logic_vector(7 downto 0);
        sel : IN  std_logic;
        S   : OUT std_logic_vector(7 downto 0);
        C   : OUT std_logic;
        N   : OUT std_logic;
        V   : OUT std_logic;
        Z   : OUT std_logic
    );
end component;

-- Test vector record
type stimulus_response_type is record
    operand_A : std_logic_vector(7 downto 0);
```

```
    operand_B : std_logic_vector(7 downto 0);
    select_in : std_logic;
    result_S  : std_logic_vector(7 downto 0);
    carry     : std_logic;
    overflow  : std_logic;
    negative  : std_logic;
    zero      : std_logic;
end record;

type stimulus_response_array_type is array(natural range <>)
   of stimulus_response_type;

constant TEST_DATA : stimulus_response_array_type := (
    ("00000010","00000001",'0',"00000011",'0','0','0','0'),
    ("00000010","00000001",'1',"00000001",'1','0','0','0'),
    ("01000000","01111000",'0',"10111000",'0','1','1','0'),
    ("01000000","01111000",'1',"11001000",'0','0','1','0'),

    ("11111111","00000001",'0',"00000000",'1','0','0','1'),
    ("11111111","11111111",'0',"11111110",'1','0','1','0'),
    ("00000000","11111111",'1',"00000001",'0','0','0','0'),

    ("01111111","00000001",'1',"01111110",'1','0','0','0'),
    ("10000000","00000001",'1',"01111111",'1','1','0','0'),
    ("00000000","10000000",'0',"10000000",'0','0','1','0'),

    ("01010101","00110011",'1',"00100010",'1','0','0','0'),
    ("11001100","00110011",'0',"11111111",'0','0','1','0')
);

begin

cycle <= not cycle after PERIOD/2;

i_addsub : addsub
    port map(
        A   => A_s,
        B   => B_s,
        sel => sel_s,
        S   => S_s,
        C   => C_s,
        N   => N_s,
        V   => V_s,
        Z   => Z_s
    );

stimuli : process
begin
    wait for PERIOD;
    wait for OFFSET;
```

```
    for I in TEST_DATA'range loop
        wait until rising_edge(cycle);
        A_s   <= TEST_DATA(I).operand_A;
        B_s   <= TEST_DATA(I).operand_B;
        sel_s <= TEST_DATA(I).select_in;

        wait for OFFSET;

        assert S_s = TEST_DATA(I).result_S report "error in
           sum" severity warning;
        assert C_s = TEST_DATA(I).carry     report "error in
           carry flag" severity warning;
        assert V_s = TEST_DATA(I).overflow  report "error in
           overflow flag" severity warning;
        assert N_s = TEST_DATA(I).negative  report "error in
           negative flag" severity warning;
        assert Z_s = TEST_DATA(I).zero      report "error in
           zero flag" severity warning;
    end loop;

    wait until rising_edge(cycle);
    wait for 3*PERIOD;

    assert false report "planned end of simulation" severity
       failure;
    wait;
end process;

end architecture behavioral;
```

Listing 8: Constraints for Addsub on MODSYS 2.0

```
##################################################################
# Addsub Unit     Constraints for MODSYS 2.0 Board
##################################################################

##############
# CONNECTOR 3
# Button     sel
set_property PACKAGE_PIN G16 [get_ports {sel}];
set_property IOSTANDARD LVCMOS33 [get_ports {sel}];

##############
# CONNECTOR 3
# LEDs     S[7:0]
set_property PACKAGE_PIN H15 [get_ports {S[0]}];
set_property PACKAGE_PIN K13 [get_ports {S[1]}];
set_property PACKAGE_PIN J13 [get_ports {S[2]}];
set_property PACKAGE_PIN K15 [get_ports {S[3]}];
set_property PACKAGE_PIN J15 [get_ports {S[4]}];
```

```
set_property PACKAGE_PIN E16 [get_ports {S[5]}];
set_property PACKAGE_PIN E15 [get_ports {S[6]}];
set_property PACKAGE_PIN H17 [get_ports {S[7]}];
set_property IOSTANDARD LVCMOS33 [get_ports {S[*]}];

##############
# CONNECTOR 3
# LEDs     Status Flags C, V, N, Z
set_property PACKAGE_PIN D17 [get_ports {C}];
set_property PACKAGE_PIN U9  [get_ports {V}];
set_property PACKAGE_PIN V5  [get_ports {N}];
set_property PACKAGE_PIN V4  [get_ports {Z}];

set_property IOSTANDARD LVCMOS33 [get_ports {C}];
set_property IOSTANDARD LVCMOS33 [get_ports {V}];
set_property IOSTANDARD LVCMOS33 [get_ports {N}];
set_property IOSTANDARD LVCMOS33 [get_ports {Z}];

##############
# CONNECTOR 4
# Switches     A[7:0], B[7:0]
# (Same mapping as Task 1)
set_property PACKAGE_PIN G18 [get_ports {A[0]}];
set_property PACKAGE_PIN B11 [get_ports {A[1]}];
set_property PACKAGE_PIN A11 [get_ports {A[2]}];
set_property PACKAGE_PIN D15 [get_ports {A[3]}];
set_property PACKAGE_PIN C15 [get_ports {A[4]}];
set_property PACKAGE_PIN A13 [get_ports {A[5]}];
set_property PACKAGE_PIN A14 [get_ports {A[6]}];
set_property PACKAGE_PIN F15 [get_ports {A[7]}];

set_property PACKAGE_PIN R7 [get_ports {B[0]}];
set_property PACKAGE_PIN R5 [get_ports {B[1]}];
set_property PACKAGE_PIN R6 [get_ports {B[2]}];
set_property PACKAGE_PIN N4 [get_ports {B[3]}];
set_property PACKAGE_PIN M4 [get_ports {B[4]}];
set_property PACKAGE_PIN T3 [get_ports {B[5]}];
set_property PACKAGE_PIN R3 [get_ports {B[6]}];
set_property PACKAGE_PIN U7 [get_ports {B[7]}];

set_property IOSTANDARD LVCMOS33 [get_ports {A[*]}];
set_property IOSTANDARD LVCMOS33 [get_ports {B[*]}];

##################################################################
# END OF FILE
##################################################################
```